
LLD – DAY 5

SOLID Design Principles

◆ Why SOLID Principles?

✗ Problems without SOLID

- Poor **maintainability**
 - Low **readability**
 - More **bugs**
 - Tight coupling
 - Difficult to extend code
-

◆ What is SOLID?

SOLID is a set of **5 design principles** that help in writing:

- Clean code
 - Maintainable systems
 - Scalable designs
-

◆ SOLID stands for

Letter	Principle
S	Single Responsibility Principle (SRP)
O	Open–Closed Principle (OCP)
L	Liskov Substitution Principle (LSP)
I	Interface Segregation Principle (ISP)
D	Dependency Inversion Principle (DIP)

✦ Today covered: S, O, L

◆ S – Single Responsibility Principle (SRP)

✓ Definition

A class should **do only one thing** A class should have **only one reason to change**

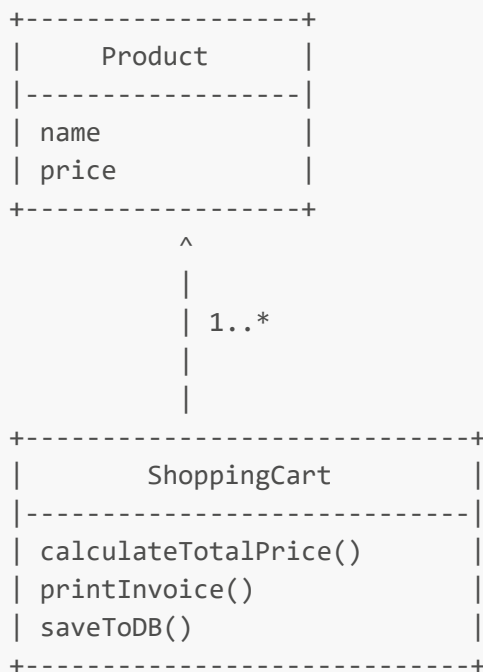
◆ Problem (Violation of SRP)

🎤 Given Scenario

- **Product** class contains:
 - price
 - name
- **ShoppingCart** class contains:
 - calculateTotalPrice()
 - printInvoice()
 - saveToDB()

👉 **ShoppingCart is doing multiple responsibilities** 👉 This **violates SRP**

🖋️ Text Diagram (SRP Violation)

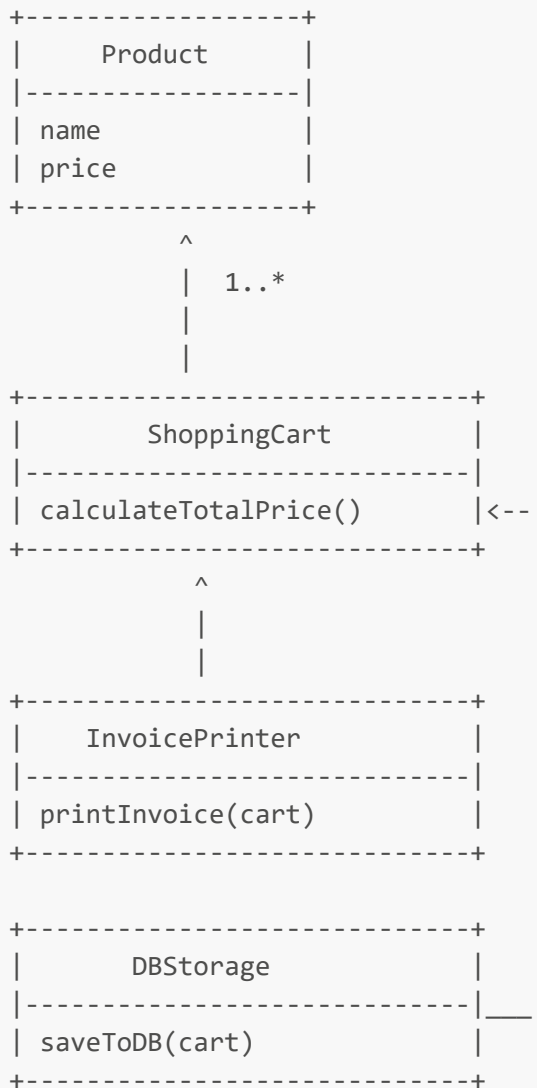


✗ ShoppingCart breaks SRP

◆ Solution (Apply SRP)

👉 Split responsibilities into **separate classes**

🖋️ Text Diagram (SRP Followed)



✓ Each class has **single responsibility**

◆ O – Open Closed Principle (OCP)

✓ Definition

A class should be **open for extension** but **closed for modification**

✦ Also called **Repair Rules Principle**

◆ Problem (OCP Violation)

🗨 Scenario

- `DBStorage` class has:

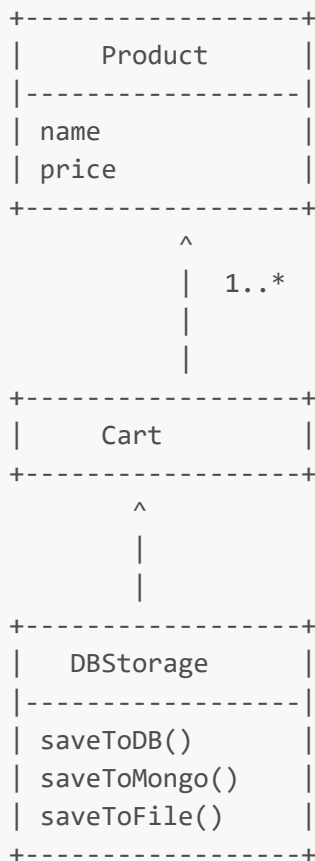
- `saveToDB()`

Later requirement:

- saveToMongo()
- saveToFile()

👉 We modify existing class 👉 **This breaks OCP**

🖋️ Text Diagram (OCP Violation)



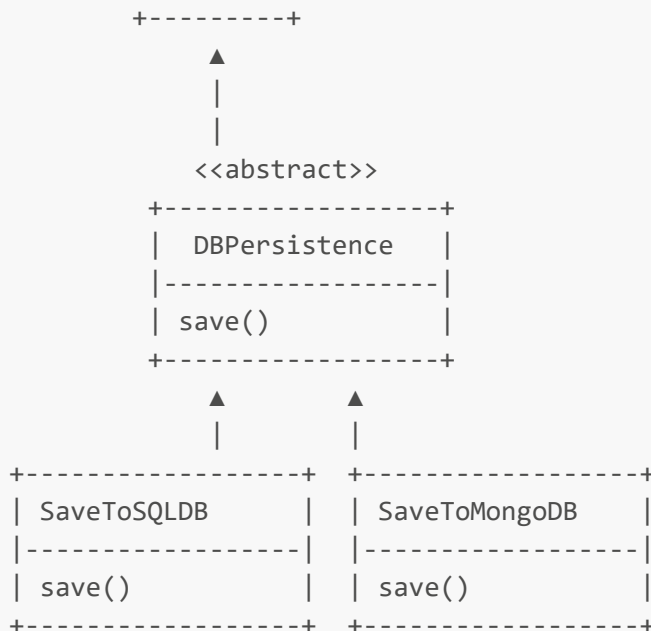
✗ Modification breaks OCP

◆ Solution (Apply OCP)

👉 Use **Abstraction + Inheritance + Polymorphism**

🖋️ Text Diagram (OCP Followed)





✓ Add new storage **without modifying existing code**

◆ L – Liskov Substitution Principle (LSP)

✓ Definition

Subclasses should be **substitutable for their base classes** without breaking the program

◆ Concept Explanation

If:

```
A* a = new B();
```

Then:

```
a->method1();
a->method2();
```

👉 Should work **exactly as base class expects**

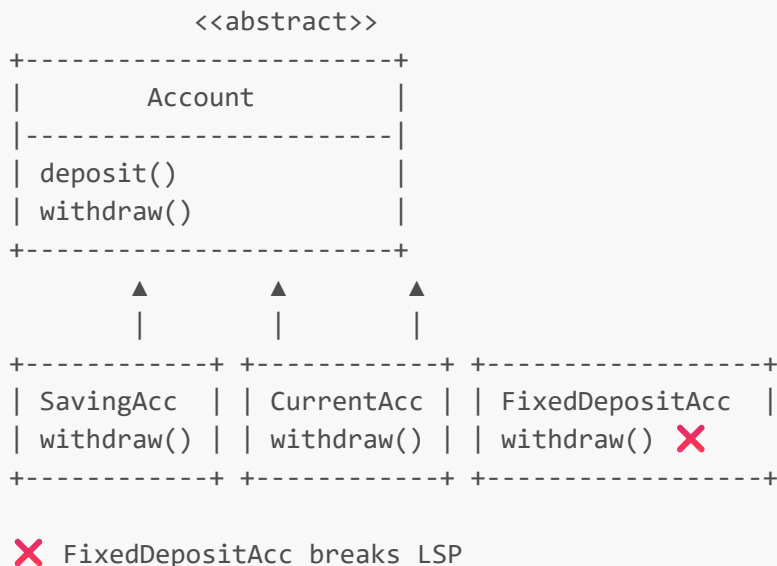
◆ Problem (LSP Violation)

💡 Example from Image

- Abstract class: **Account**
 - deposit()
 - withdraw()
- Subclasses:
 - SavingAccount
 - CurrentAccount
 - FixedDepositAccount ❌

👉 FixedDepositAccount **does not support withdraw()** 👉 Throws exception 👉 **Breaks LSP**

✏️ Text Diagram (LSP Violation)



◆ Naive Solution (WRONG)

```

if(acc is FixedDeposit)
    deposit();
else
    withdraw();

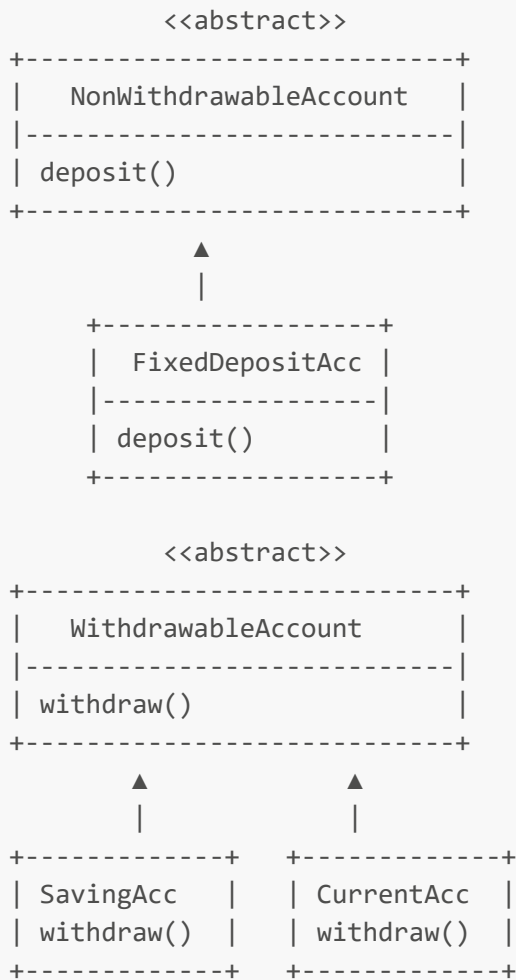
```

❌ Client becomes tightly coupled ❌ Breaks **OCP**

◆ Correct Solution (Apply LSP)

👉 Split abstraction properly

✏️ Text Diagram (LSP Followed)



✓ All subclasses are safely substitutable ✓ LSP satisfied



DAY 5 SUMMARY

✓ SRP → One class, one responsibility ✓ OCP → Extend, don't modify ✓ LSP → Subclass must behave like base class ✓ SOLID improves maintainability ✓ Core of LLD interviews



Interview Golden Lines

- **SRP reduces reasons to change**
- **OCP avoids modification of tested code**
- **LSP ensures polymorphism safety**
- **SOLID makes code scalable**